

Lachesis: Lifetime-Aware KV Cache Placement for Agent Serving across HBM and High-Bandwidth Flash

Jaehoon Yang
Seoul National University
Seoul, Republic of Korea
jaehoon.yang@snu.ac.kr

Jeongmin Lee
KAIST
Daejeon, Republic of Korea
jeongminl@kaist.ac.kr

Haneul Park
University of Illinois,
Urbana-Champaign
Urbana, IL, USA
hnpark2@illinois.edu

Seung Yul Lee
Seoul National University
Seoul, Republic of Korea
trionphant1@snu.ac.kr

Nam Sung Kim
University of Illinois,
Urbana-Champaign
Urbana, IL, USA
nam.sung.kim@gmail.com

Jae W. Lee
Seoul National University
Seoul, Republic of Korea
jaewlee@snu.ac.kr

Abstract

Large language model (LLM) serving is increasingly dominated by agentic workloads, in which agents and their sub-agents accumulate context as KV cache across many requests, consuming substantial memory. High-bandwidth flash (HBF) is a promising solution, providing an order of magnitude greater capacity at HBM-class read bandwidth, but its finite write endurance is the key limiting factor. Our key insight is that KV cache should be placed across HBM and HBF by its lifetime. Placing shorter-lived data in HBM lets HBM absorb more of an agent run’s writes and sends less of them to HBF. As the lifetime of KV cache in agentic serving is dictated by the harness, the program that orchestrates the agents, we analyze its behavior and identify three axes along which lifetime diverges, temporal, structural, and inter-worker. Guided by these observations, we present Lachesis, a lifetime-aware KV cache placement layer between the agent harness and the serving engine. At write time, it places each segment in HBM or HBF according to its lifetime, and frees its blocks once the segment is no longer read. In trace-driven simulation, Lachesis extends HBF lifetime by 1.19–3.13× over HBM-first placement, reaching 3.3–12.2 device-years. Even under continuous 24×7 operation at the full load a tight SLO admits, HBF outlasts its five-year warranty on the multi-agent trace.

1 Introduction

LLM serving is shifting toward agentic workloads, in which an agent works on a single user task over many LLM requests and spawns sub-agents that run concurrently alongside it [55]. Whereas a conventional request holds one context for the duration of one request, an agent holds the context it has accumulated so far in the session as KV cache, and that KV cache can stay live well beyond the request that wrote it, in some cases until the session ends (Figure 1, top left). Together with the sub-agents a session spawns, this longer lifetime pushes the volume of KV cache live at one time far beyond that of conventional serving. The pressure

this KV cache puts on an accelerator’s high-bandwidth memory (HBM) forces a deployment to add accelerators or serve fewer sessions at once, either of which raises the hardware cost of a served request [14, 23, 41].

High-bandwidth flash (HBF) is emerging as a promising class of memory for LLM inference, offering an order of magnitude greater capacity than HBM with HBM-class sequential read bandwidth [38]. That capacity absorbs the added pressure and brings the cost of a served request back down. The key limiting factor for HBF, however, is write endurance. NAND flash tolerates only a finite number of program/erase (P/E) cycles, so a DRAM-like role that absorbs a large volume of writes can wear it out in a fraction of HBF’s required service life. Early HBF designs therefore confine HBF to read-only data such as model weights and shared-prefix KV caches [14, 20, 29]. As KV caches are a dominant source of serving-time memory pressure in agentic and large-batch workloads [25, 26, 36], recent works extend HBF to dynamically generated KV cache and select what to write to it, mainly by its read-to-write ratio [23, 33, 47].

Unlike prior work, which selects KV cache for HBF by its read-to-write ratio [23], we select it by its lifetime. When HBF is used alongside HBM, every KV block a session writes lands in one of the two tiers, and only the bytes that land in HBF spend endurance. HBF therefore lasts longer the more of a session’s writes HBM absorbs. The volume HBM absorbs over a period is its capacity multiplied by the number of times each block is reclaimed when the KV cache it holds is discarded. Placing shorter-lived KV blocks in HBM therefore lets HBM absorb more of a session’s writes and sends less of them to HBF, which requires knowing a block’s lifetime before it is written.

In agentic serving, KV lifetime is set by the agent harness, the program that orchestrates the session [5, 24, 37, 46], so we analyze how the harness manages it. Depending on how it structures the session and the sub-agents it spawns, the harness keeps some KV cache for the whole session and

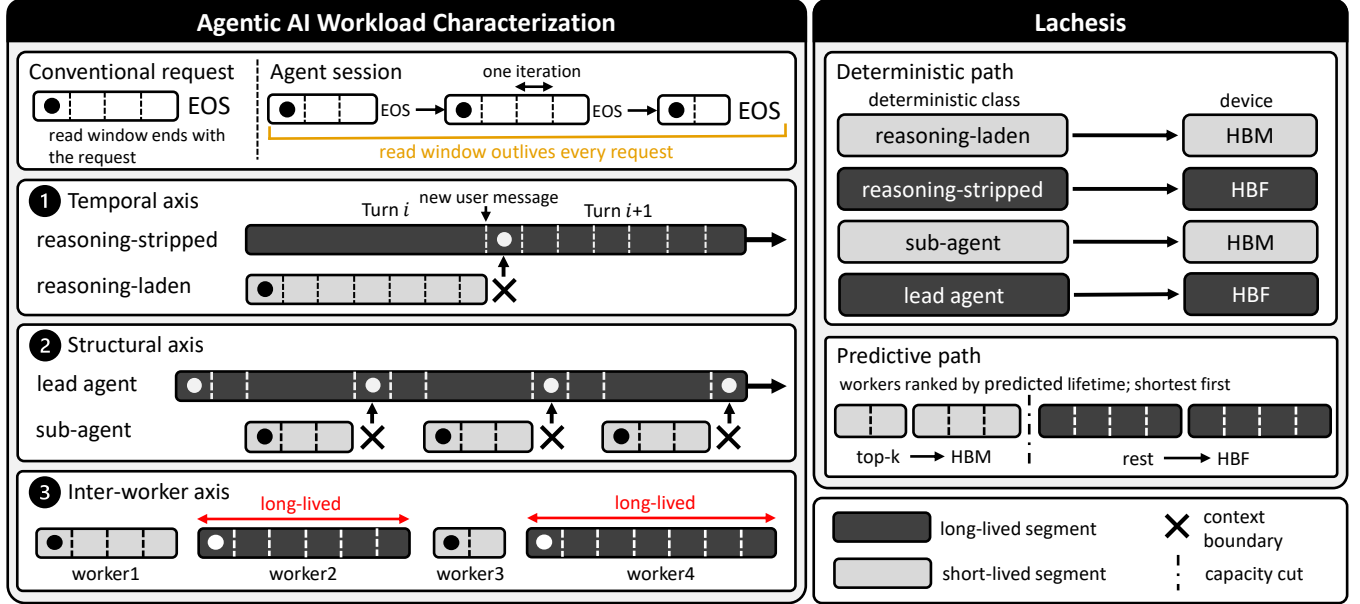


Figure 1. Top left: the read window of a conventional request, and of an agent session. Left: the three axes along which context-segment lifetimes differ. Right: Lachesis’s two placement paths.

discards the rest partway through. It makes that decision for KV in groups, so each group shares one lifetime. We call such a group a *context segment* and the point at which it is discarded a *context boundary*.

Our analysis shows that segment lifetimes diverge along three axes of session structure, shown in the left panel of Figure 1. The *temporal* axis follows the *turns* of a session, one opened by each user message. When the next message arrives, the harness strips the reasoning generated within the turn and carries only its output forward. The reasoning segment holds 66–83% of the KV the turn writes yet lives only one turn, whereas the output segment lives 7.6–17.2× longer. The *structural* axis follows delegation within a turn. A *lead agent* may, depending on the harness, delegate subtasks to *sub-agents*, each running in its own context. When a sub-agent returns, the harness discards its segment and keeps only its result. Sub-agent segments produce most of the KV written within the turn, while the lead-agent segment lives 4.0–6.1× longer. The temporal and structural boundaries are both predetermined and known to the harness. On those two axes the short-lived segment, which takes large portion of writes, can therefore be placed in HBM at the write and freed the moment the boundary arrives. The third axis, inter-worker, follows the *workers* that carry out those subtasks, whose lifetimes differ by when each completes. That completion time is not fixed in advance, and worker lifetimes spread by 3.7–23.6× between p10 and p90, so segments of one class carry significantly different lifetimes. Our key observation is that their relative order can be predicted at spawn time from the record the harness uses to build each

worker’s context. Placement can thus be decided before the KV is written along all three axes.

We present Lachesis, a lifetime-aware KV cache placement layer that decides which KV reaches HBF before a segment is written (§4). Lachesis places along two paths (Figure 1, right). The *deterministic path* covers the temporal and structural axes, where the harness knows whether a segment is discarded at the boundary or retained past it. A short-lived context segment has priority for HBM, and a long-lived context segment is placed in HBF. The *predictive path* covers the inter-worker axis, where no context boundary separates one worker’s segment from another’s. Lachesis predicts each segment’s lifetime at spawn, ranks the segments shortest-first, and admits as many as HBM has room for, placing the rest in HBF. Lifetime is tracked entirely in Lachesis, and the serving engine sees it only through two calls. The first allocates a KV block from the tier Lachesis specifies, HBM or HBF, so every placement decision of either path is issued as one allocation. The second frees the blocks of a segment whose lifetime has ended, at the boundary the harness reports. Across agent traces replayed through a simulated two-tier memory system, Lachesis extends HBF lifetime by 1.19–3.13× over HBM-first placement, which allocates in HBM while it has a free block and in HBF otherwise (§5). At a tight service-level objective (SLO) under continuous 24×7 operation, HBF outlasts a five-year warranty on the multi-agent trace.

In summary, this paper makes the following contributions:

- We show that the context an agent session accumulates divides into segments of differing lifetime along three axes, temporal, structural, and inter-worker, and

that a segment’s lifetime can be determined before it is written.

- We design Lachesis, a lifetime-aware KV cache placement layer between the agent harness and the serving engine. Its deterministic path places the segments the harness marks as short-lived in HBM, and its predictive path admits the workers predicted to be shortest-lived, so that less KV reaches HBF and its endurance lasts longer.
- We evaluate Lachesis against HBM-first placement and show that it extends HBF lifetime by 1.19–3.13×, and that HBF then outlasts a five-year warranty on the multi-agent trace at a tight SLO. A session writes the same KV volume under either policy; Lachesis admits the shortest-lived segments to HBM and frees each at its boundary, so the same HBM capacity absorbs more of that volume and less of it reaches flash.

2 Background and Motivation

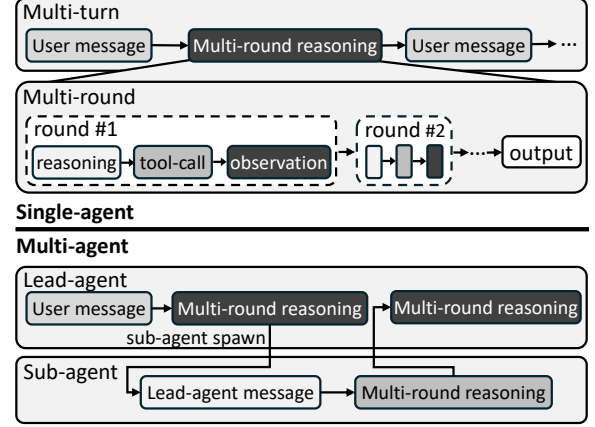
2.1 Agent Serving Intensifies the Capacity Wall

Agentic serving is increasingly constrained by a *memory-capacity wall*, pushing up the cost of serving each request. The pressure grows from two sides, the model footprint and the KV cache. The model footprint has grown faster than per-accelerator HBM capacity, requiring more accelerators simply to host a model [14, 41]. The other side is the KV cache, which grows with the context each session accumulates.

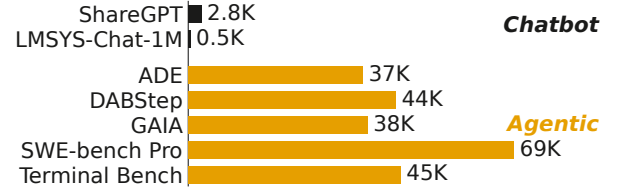
An agent session accumulates context by repeating the reason-act-observe loop of a ReAct-style agent until the task is done (Figure 2(a), top). Each pass is a *round*, a single LLM request, and it appends all three to the context every later round must read [52, 55]. A session divides into *turns*, one per user message, and a turn holds one or more rounds. An observation is sized by the environment rather than by a person, so a source file or a command’s output enters the context whole. The peak context lengths of agentic workloads are 29× larger than those of conventional chatbot workloads (Figure 2(b)).

A multi-agent session holds more than one such context at a time (Figure 2(a), bottom). A *lead agent* carries the user’s task and delegates a subtask to a *sub-agent*, which runs its own rounds in a context of its own and returns a result [4, 5, 21]. The session therefore occupies the lead agent’s context and one more for every sub-agent running under it. Many such sessions are served at once, and each holds its accumulated context in HBM. Deployments must either spread each request across more accelerators or reduce the number of concurrently resident contexts, both of which increase the hardware cost per served request [14, 23, 41].

Existing KV-cache optimizations reduce memory demand, but do not eliminate the capacity wall. Quantization and compression reduce the bytes per cached token only by a



(a) Execution structure of an agent session, single-agent above and multi-agent below.



(b) Mean peak context per conversation or agent task.

Figure 2. Context accumulation in agent serving.

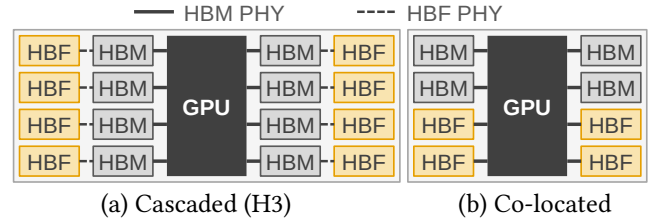


Figure 3. GPU architectures equipped with HBF.

constant factor, while KV capacity still grows linearly with context length [26, 28]. Eviction and a bounded attention window cap the KV footprint by discarding past KV, which degrades accuracy on long-context workloads [48, 57]. Sparse attention reduces the KV read per step, but the KV footprint remains unchanged [43]. Thus, serving long agentic contexts still exceeds HBM capacity, demanding a substantially larger memory tier.

2.2 High-Bandwidth Flash: Capacity Opportunity, Endurance Constraint

HBF, an emerging stacked memory built from NAND flash, can relieve the memory-capacity wall while providing HBM-class sequential read bandwidth. HBF is projected to provide 16× the per-stack capacity of HBM, and to exploit thousands of NAND planes to deliver high sequential bandwidth

Table 1. The three characterization axes. The labels name the two sides of each axis’s own comparison, not one threshold across axes.

Axis	Scope relationship	What creates the split	Short-lived	Long-lived	Signal
Temporal (§3.3)	session $\supset$ turn	Reasoning stripping	reasoning-laden context segment	reasoning-stripped context segment	Deterministic boundary
Structural (§3.4)	session $\supset$ sub-agent	Sub-agent return	sub-agent segment	lead-agent segment	Deterministic boundary
Inter-worker (§3.5)	across worker invocations	Variation in return time	Early-returning worker’s segment	Late-returning worker’s segment	Predicted rank

Table 2. Execution regimes. Each is configured so that exactly one context boundary is active. Both regimes use Gemma 4.

Regime	Harness	Benchmarks	Session shape	Active boundary	Axes
Temporal	Harbor, OpenAI message format, model’s own chat template	τ^3 -bench [39, 51], BFCLv4 multi-turn [35]	Multi-turn, single agent	Turn-end reasoning stripping	§3.3
Structural	AOrchestra (inherits Claude Code)	AppWorld [44], DeepSearchQA [13], SWE-bench Verified [17], TerminalBench [27], WebArena [60]	Single-turn, multi-round lead-agent/sub-agent	Sub-agent return	§3.4, §3.5

[38, 40]. Its larger capacity allows more contexts to remain resident on each accelerator, while its read bandwidth is sufficient to serve attention efficiently.

HBF is therefore a promising alternative to HBM, and Figure 3 shows two architectures explored by prior works that directly integrate HBF with accelerators. Daisy-chained designs connect HBF stacks behind HBM, allowing HBF traffic to reach the accelerator through the HBM stacks (Figure 3(a)) [14]. Co-located designs instead attach HBF directly alongside the accelerator (Figure 3(b)) [23, 41]. When accessing HBF-resident data in co-located designs, the accelerator can utilize the aggregate memory bandwidth of all stacks through direct and inter-stack paths [3, 20].

Prior works have addressed the main performance limitations of HBF, which are outside the scope of this work [14, 20, 29, 47]. First, HBF read latency is orders of magnitude higher than HBM. LLM inference exposes predictable sequential accesses that enable aggressive prefetching to hide this latency. Second, sustaining HBF’s peak read bandwidth requires accessing many flash planes in parallel; prior work achieves this through plane-balanced data layouts.

The remaining obstacle to using HBF in LLM serving is the limited write endurance. Because NAND supports only a finite number of P/E cycles, prior HBF designs have largely focused on read-only state that is written once and then repeatedly read [38]. Designs such as H3 place model weights and precomputed KV cache in HBF while retaining online-generated KV in HBM [14, 20, 29].

For agentic serving, however, the generated KV becomes the dominant capacity bottleneck, making write endurance a first-order challenge. Unlike model weights, whose footprint is fixed once the model is loaded, generated KV grows with context length and the number of resident sessions (Figure 2(b)). Fully exploiting HBF, therefore, requires placing the generated KV in HBF [23, 41]. The KV cache is generally well suited to NAND flash. Each entry is written once during prefill or decode and then repeatedly read by subsequent attention steps [22]. But short-lived KV with little reuse can nonetheless quickly exhaust write endurance [41].

2.3 Why Agent Sessions Require Lifetime-Aware KV Cache Placement

Although Kyung et al. propose placing the KV cache with a high read-to-write ratio in HBF [23], this criterion does not directly apply to agentic serving. They show that output length largely determines the read-to-write ratio because longer generation reuses the KV cache across more decode steps. This relationship holds for stateless requests, where all KV is created and consumed within a single request and therefore shares the request’s lifetime. Agent sessions are stateful, however, and different parts of their context can remain useful for different lengths of time. The harness may discard some context while retaining the rest for later turns or subtasks. We call a group of tokens that share the same lifetime a *context segment*.

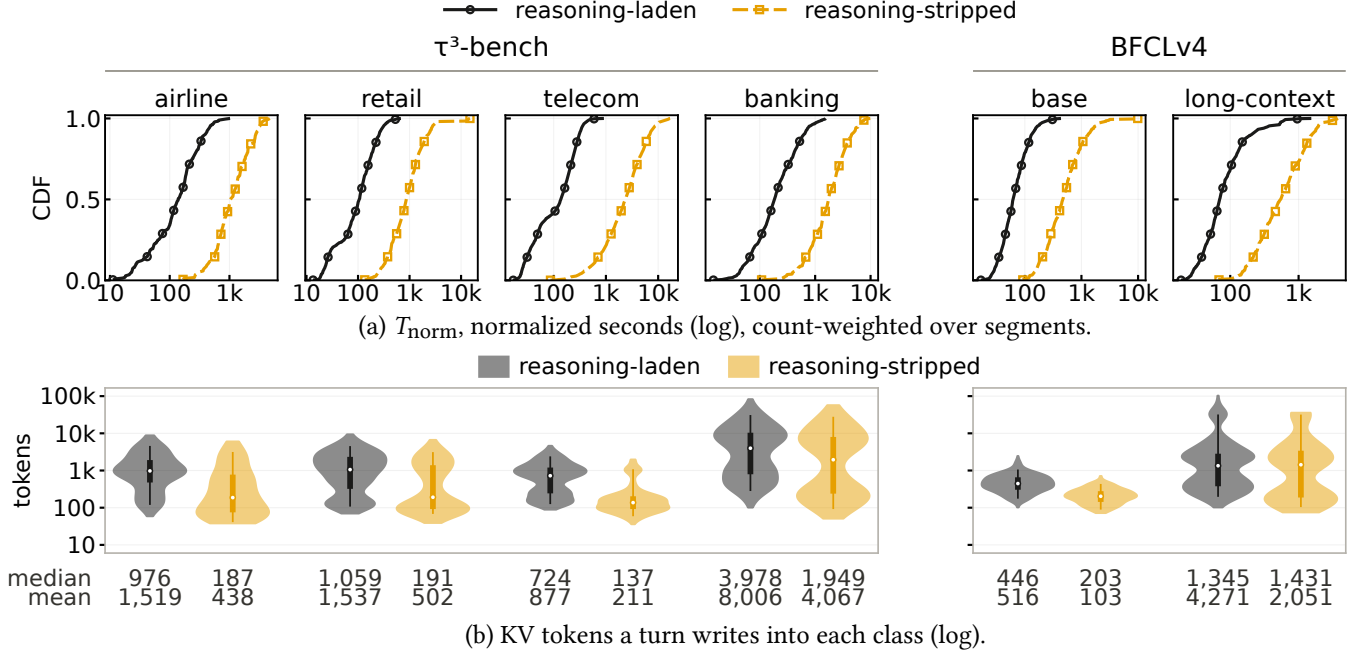


Figure 4. Temporal axis, per domain. T_{norm} is drawn at one sampled user delay, median 120 s, and one sampled tool-call latency, mean 0.69 s.

HBF placement should therefore be decided per context segment rather than per request or session. Keeping short-lived segments in HBM reduces the write traffic reaching flash, thereby extending HBF lifetime. In the following section, we analyze agentic serving workloads to identify the sources of lifetime variation and characterize which context segments are short- or long-lived.

3 Context-Lifetime Analysis for Agent Serving

3.1 Analysis Overview

Table 1 summarizes the three characterization axes that arise in agentic serving, where each axis separates short-lived from long-lived context segments. Along the temporal axis, reasoning context is discarded at the turn end, and the remaining reasoning-stripped context merges with the session context (§3.3). Similarly, along the structural axis, sub-agent context excluding the result is discarded at the sub-agent return (§3.4). The discarded segment is a short-lived context segment, and what survives the boundary is a long-lived context segment. Along the inter-worker axis, the class is constant and worker segments differ in lifetime by when each worker returns (§3.5).

The harness fixes a segment’s class before the segment is written, so the lifetime is known at the write. The first two splits are deterministic, fixed at the turn end and at sub-agent spawn. The last split, worker lifetime, is predictable from the metadata the harness records when it spawns each worker.

A reactive policy such as least-recently-used (LRU) cannot assign the class before observing subsequent accesses and would require migration to correct the placement later.

3.2 Methodology

We isolate the turn end and sub-agent return in two execution regimes, each configured so that only the context boundary under study can close a read window (Table 2). A lifetime difference measured in a regime is therefore attributable to that regime’s boundary. The temporal regime runs on Harbor [15] and the structural regime on AOrchestra, which builds each sub-agent from an explicit $\langle \text{Instruction, Context, Tools, Model} \rangle$ specification at spawn [37]. We call that specification the *spawn record*, from which §3.5 ranks worker lifetimes. No harness records segments, so they are recovered from a per-request trace. The trace records the exactly matched input-prefix length and the cache read SGLang reports [58]. A segment’s read window closes where the segment stops appearing in the context that later requests reuse, or at the end of the session if no boundary closes it. Rematerialization produces a new segment, not an in-place rewrite, so each segment is materialized once with its own write volume.

The primary metric, normalized context lifetime, is how long a segment must remain readable on a fixed reference clock rather than on server wall time:

$$T_{\text{norm}}(s) = G_{\text{dec}}(s) \text{TPOT}_{\text{ref}} + N_{\text{obs}}(s) \text{TTFT}_{\text{ref}} + \Lambda_{\text{tool}}(s) + \Lambda_{\text{user}}(s) \quad (1)$$

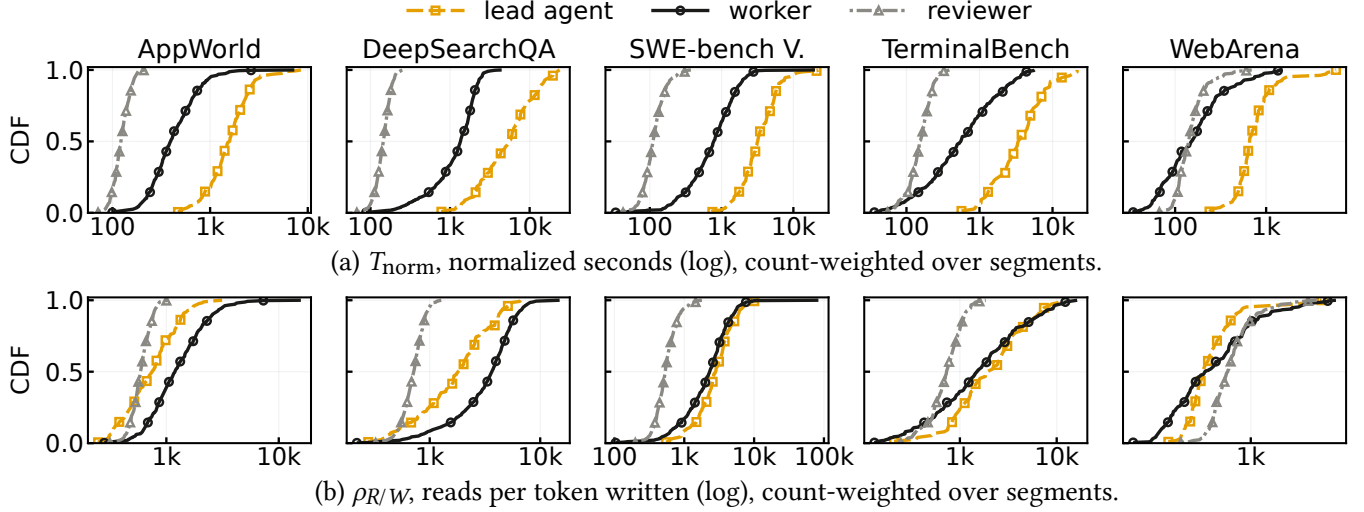


Figure 5. Structural axis, per benchmark. T_{norm} is drawn at one sampled tool-call latency per tool class.

Every term is counted while segment s is live: $G_{\text{dec}}(s)$ is the number of tokens generated on the decode node, $N_{\text{obs}}(s)$ the number of observation prefills, $\Lambda_{\text{tool}}(s)$ the tool-call latency, and $\Lambda_{\text{user}}(s)$ the user delay before the next message. The clock is the decode node’s clock in a prefill–decode disaggregated deployment [10, 34, 59]. Its reference point is a time per output token $\text{TPOT}_{\text{ref}} = 0.2 \text{ s}$ [8, 18, 19] and a time to first token $\text{TTFT}_{\text{ref}} = 1.0 \text{ s}$ [7, 56]. Normalizing removes decode batching, server load, and queueing delay, which are properties of the serving deployment rather than of the agent session. $\Lambda_{\text{tool}}(s)$ and $\Lambda_{\text{user}}(s)$ are modeled rather than taken from the trace, because tool-call latency and user delay depend on the environment an agent is deployed in (§5.2). The read-to-write ratio $\rho_{R/W}(s)$, by which prior work selects KV for HBF [23], divides the KV read from a segment by the KV written to materialize it. Each session is analyzed independently, and no measurement aggregates across concurrent sessions.

3.3 Temporal Axis: Reasoning-Content Stripping

Along the temporal axis, reasoning determines how long a context segment remains live. Across multiple rounds within a turn, the harness accumulates reasoning context across tool calls along with non-reasoning context [9, 32]. The accumulated context is read throughout the later rounds in the same turn [55]. When the turn ends as the next user message arrives, the harness discards the reasoning context, the reasoning-laden context segment, and only the remaining reasoning-stripped context segment from the previous turn is passed to the new turn [31].

Figure 4(a) shows that the reasoning-stripped context segment outlives the reasoning-laden context segment by a median ratio of 7.6–17.2 $\times$ across all domains. The reasoning-stripped context segment is retained across multiple con-

secutive turns, while the reasoning-laden context segment is retained only during the single turn that generates the context. The lifetime distributions of the two segments are clearly separated: a separability, the probability that a random reasoning-stripped context segment outlives a random reasoning-laden context segment, is 0.96–0.99.

Placing short-lived reasoning-laden context segment in HBF is particularly costly because it is much larger than the reasoning-stripped context segment. Of the tokens written by a turn, 66–83% belong to the reasoning-laden context segment (Figure 4(b)). These tokens consume extensive HBF P/E cycles even though the harness soon discards them at the next turn end. The same write budget can instead retain many smaller, long-lived contexts in HBF.

Takeaway 1

- The reasoning-laden context segment is short-lived yet holds 66–83% of the tokens a turn writes, while the reasoning-stripped context segment that replaces it is long-lived.
- HBM should hold the short-lived context segment and HBF the long-lived context segment.

3.4 Structural Axis: Lead-Agent/Sub-Agent Isolation

In multi-agent sessions, the lead-agent context is retained longer than the context created for each delegated subtask. The lead agent delegates a subtask to a sub-agent, which executes the task in a separate context. When the sub-agent returns, only its result is merged into the lead-agent segment, while the remaining sub-agent segment is discarded. This separation is by design of the multi-agent execution model, not an optimization specific to one harness [4, 5, 21, 24, 30]. Therefore, lead-agent segment contains session-level context, including the system prompt, user objective, spawn

records, and returned sub-agent results, and sub-agent segment contains the context of a single sub-agent, including its prompt, spawn instruction, and generated content. While lead-agent segment remains available until the session ends, sub-agent segment is used only until that sub-agent returns.

The sub-agent segment is consistently shorter-lived than the lead-agent segment across all benchmarks. The lifetime of sub-agent segment varies by role: a worker actively executes a delegated task through multiple rounds of tool use, while a reviewer receives a completed worker trace and produces a review without further delegation. Figure 5(a) shows that the lead-agent segment outlives a worker’s segment by a median ratio of 4.0–6.1 $\times$ and a reviewer’s by 4.7–38.3 $\times$, with separability 0.90–1.00 between their lifetime distributions.

For HBF, placing the long-lived lead-agent segment is beneficial although its reuse is comparable to that of a worker sub-agent. After the lead agent delegates the subtasks, sub-agent decoding, observation prefills, and tool-call latency increase the lead agent’s T_{norm} while the lead-agent segment remains unread. This waiting period accounts for 71–88% of the lead-agent lifetime. As a result, the lead agent and worker overlap in $\rho_{R/W}$ by 0.25–0.60 despite their large difference in lifetime (Figure 5(b)). Reuse should therefore not be used as a proxy for lifetime when deciding HBF placement.

Takeaway 2

- The sub-agent segment is short-lived, while the lead-agent segment is long-lived yet unread for 71–88% of that lifetime.
- HBM should hold the sub-agent segment and HBF the lead-agent segment.

3.5 Inter-Worker Axis: Lifetime Heterogeneity and Spawn-Time Predictability

All worker segments belong to the sub-agent class, yet their lifetimes vary widely, and no context boundary separates them. Within a single benchmark, worker T_{norm} varies by 3.7–23.6 $\times$ from p10 to p90 (Figure 6(a)), and worker write volume varies by 2.8–22.5 $\times$ over the same percentiles. The two correlate at Kendall’s τ of 0.51–0.72 per benchmark (Figure 6(b)), so a worker that writes more usually holds a longer-lived segment. A session runs many sub-agents, so the sub-agent class accounts for most of what a session writes, and treating it as homogeneous ignores that within-class spread.

Lifetime and write volume both scale with a worker’s round count, but only lifetime also includes tool-call latency. A worker’s T_{norm} is a round count times a per-round cost, which Equation 1 splits into a generation-side term, $G_{\text{dec}}(s)\text{TPOT}_{\text{ref}} + N_{\text{obs}}(s)\text{TTFT}_{\text{ref}}$, and a tool-call latency term, $\Lambda_{\text{tool}}(s)$. Write volume grows with the same round count, so the two spreads share that factor. Tool-call latency adds no tokens, so a waiting worker advances its T_{norm} without adding to its write volume. This latency differs

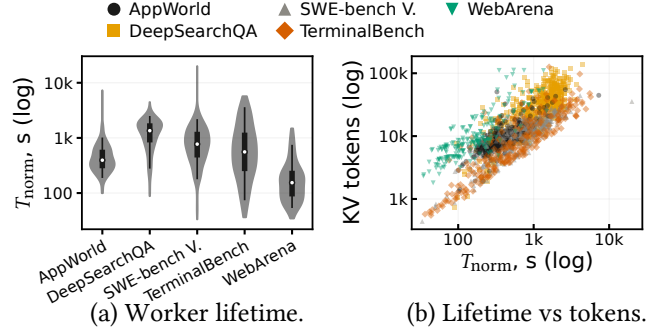


Figure 6. Worker heterogeneity, per benchmark.

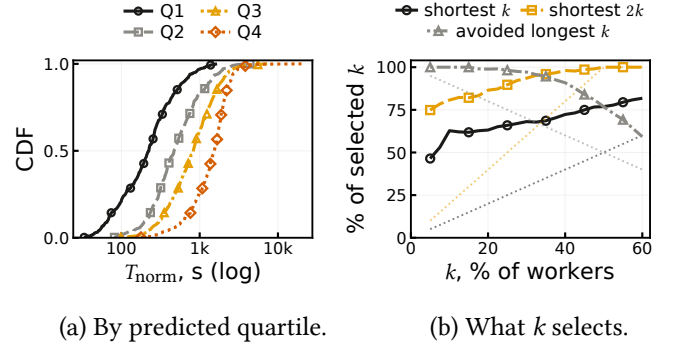


Figure 7. Spawn-time lifetime prediction.

greatly from tool to tool, so workers that write similar volumes can still have very different lifetimes. Published measurements put mean call durations three to four orders of magnitude apart, and tool execution takes up to nearly a third of end-to-end agent latency [55].

A worker’s lifetime and write volume are known only after it finishes, so we predict its lifetime rank at spawn. The target is the rank rather than the value, because estimating T_{norm} itself is hard while recovering the order is not. Analysis of a stateless request identifies that request’s output length as the signal [23]; the quantity ranked here is the worker’s whole lifetime. The predictor reads the spawn record’s instruction, context and tool list, together with where the spawn falls in the session; we call that set the worker’s *spawn signal*. The predictor is a linear model over that signal, fitted by ridge regression and read only for the order it induces. Each element enters the model as a length, an indicator per tool, a session position, or a hashed bag of its words. A single predictor serves all five benchmarks.

The predicted ranking orders workers by lifetime at Kendall’s τ of 0.53, a metric that scores the order alone. The predictor is scored by five-fold cross-validation that keeps all workers of a session in the same fold, so no worker is scored against training data from its own session. Figure 7(a) groups worker T_{norm} by predicted rank, from Q1 shortest to Q4 longest, and the four quartiles separate across the whole

range. Figure 7(b) takes the k shortest-predicted workers at each k and compares three shares of that selection with a random selection of the same size, the share among the k shortest-lived, the share among the $2k$ shortest-lived, and the share that avoids the k longest-lived. At one worker in ten, the selection holds 62.8% of the shortest-lived tenth and none of the longest-lived tenth.

Takeaway 3

- Within the worker role, lifetime varies by 3.7–23.6× from p10 to p90, and no context boundary separates that spread.
- Workers are ranked by a lifetime predicted at spawn, and the segments ranked first have priority for HBM.

On the temporal and structural axes, the segment class the harness fixes is available when the segment is written. Among workers, where no context boundary separates them, the spawn record exposes candidate signals for the variation inside that one class.

4 Lachesis Design

Based on the three axes of §3, we present Lachesis, a lifetime-aware KV cache placement layer that decides which KV reaches HBM before a segment is written. As Figure 8 shows, Lachesis lies between an agent harness such as the OpenHands SDK, AOrchestra or Claude Code [5, 37, 46] and a serving engine such as vLLM or SGLang [22, 58]. Lachesis receives three things from the harness: the id of the segment that each LLM request extends, the spawn record written at each sub-agent spawn, and the context boundary that closes a segment. It holds three components, a deterministic path that reads the segment class that the harness fixes on the temporal and structural axes, a predictive path that ranks workers by a lifetime predicted at spawn on the inter-worker axis, and a segment table that carries one row for each live segment. It reaches the serving engine as two calls, one that allocates a block in a named tier and one that frees the blocks of a closed segment. Its placement covers KV only, leaving §2.2’s read-only mapping for model weights unchanged and activations in HBM.

The deterministic path places the short-lived context segment in HBM and the long-lived context segment in HBF, by the segment class that the harness fixes. In a single-agent session, the harness reports one context boundary, the turn end (§3.3). Until it arrives, every request extends the turn’s reasoning-laden context segment, which Lachesis places in HBM. At the boundary, the harness discards that segment and materializes the reasoning-stripped context segment that carries into the next turn, which Lachesis places in HBF. In a multi-agent session, the harness also reports each sub-agent spawn, and the return that closes the sub-agent’s segment

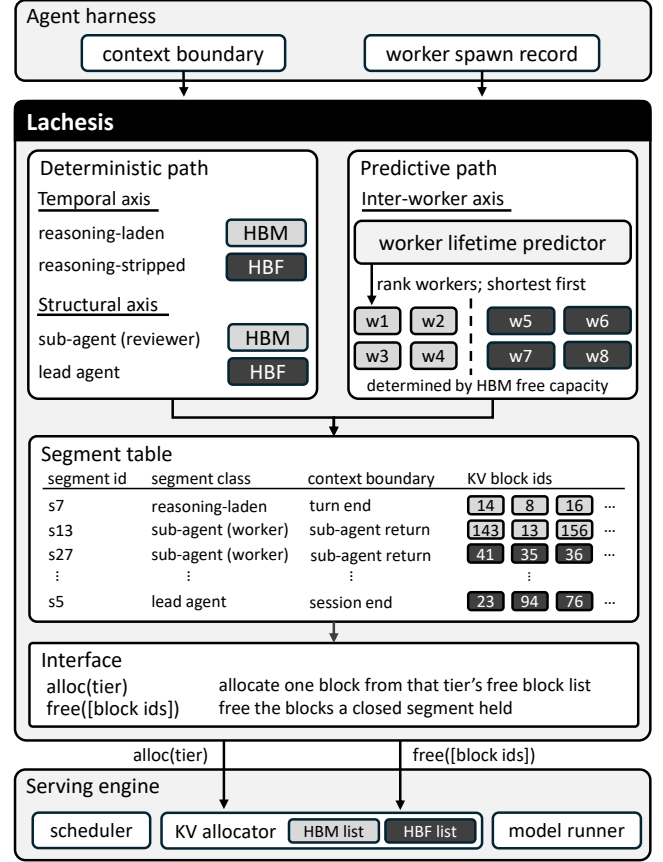


Figure 8. Overview of Lachesis.

(§3.4). The spawn separates the sub-agent segment from the lead-agent segment, so from that point Lachesis places what the sub-agent writes in HBM and what the lead agent writes in HBF. Of the two sub-agent roles, only the reviewer, whose segments are the shorter-lived, takes that placement unconditionally; a worker is admitted by the predictive path.

The predictive path ranks the workers of every session that the node serves by a lifetime predicted at their spawn, and admits the first k to HBM, allocating the rest in HBF. In a multi-agent session, every worker’s segment carries the same class, because no context boundary separates one from another (§3.5). They all close at sub-agent return and differ only in when that return comes, so the deterministic path cannot distinguish them. At each worker spawn, Lachesis scores that worker’s segment from its spawn signal, and the segment’s place in the ranking decides its memory tier. A reviewer is spawned only after the worker that it reviews, so Lachesis reserves HBM capacity for reviewers before it counts k . k is determined by HBM’s free capacity rather than set as a parameter, so a fuller HBM admits fewer workers. Only a worker’s rank among the predictions is consumed, never a lifetime, so the predictor must recover the order rather than estimate how long a worker’s segment lives.

Scoring the signal is one evaluation of a linear model, so it adds no delay before the worker starts.

Every memory tier decision is made in Lachesis, which holds one segment table row for each live segment. The row is keyed by the segment id and gives the segment class, the context boundary that closes it, and the engine’s own identifiers for the KV blocks that it holds. Lachesis adds a row at a segment’s first write and records in it the id of every block it allocates for that segment. When the harness reports the boundary that a row names, Lachesis frees that row’s blocks and removes the row. The table’s size is proportional to the number of blocks live at any moment. Lachesis names the tier for every block it places, so it tracks how much KV each tier holds.

The engine executes those decisions without representing a lifetime. Lachesis reaches the engine as two calls that it issues itself, and neither names a turn, a sub-agent, or reasoning content. `alloc(tier)` draws one KV block from the named tier’s free list, and `free([block ids])` returns the blocks that a closed segment held, each to the free list it came from. That second call fires at a context boundary, not after the inactivity that a recency policy waits for [45]. A deployed engine keeps a single free block list. Supporting a heterogeneous memory costs it one free list per tier, an HBM list and an HBF list, and nothing else. A block’s tier is therefore the free list it was drawn from. When HBM’s free list is empty, the allocation is served from the HBF free list.

5 Evaluation

5.1 Implementation

We implement Lachesis on a simulated serving engine and two-tier memory, and replay real agent traces through it. Frontier supplies the engine [11]: a scheduler and request queue, a KV-cache manager with a block-hash prefix cache, memory capacity expressed as a KV-block budget, and context that grows per round. LLMsimulator supplies the hardware [1], with the HBF extension prior work used [23, 41]: compiled operator kernels and roofline memory pricing. Its memory system holds KV in both tiers and prices each read against the bandwidth of the tier it is served from. KV bound for HBF is buffered across the decoder layers and flushed once at the end of the decode step, so HBF’s write bandwidth prices that flush rather than any decoder layer. The two meet at the operator-cost call, where the engine passes an operator’s shape and its KV length and receives its execution time on the device. Its workload generator now replays an agent run as per-request rows that name the segment each request extends. Its KV allocator now keeps a free list per tier and serves the two calls of §4.

5.2 Experimental Setup

Testbed. Lachesis is evaluated at the decode node of a prefill-decode disaggregated deployment [10, 34, 59], where the

HBF-equipped GPUs reside [23, 41]. Four GPU-memory configurations split the eight stack slots of the GPU used by Son et al. [41] between HBM and HBF, 1/7, 2/6, 3/5 and 4/4. A fifth is H3’s daisy chain, labelled 8/8* [14]. The two tiers have the same read bandwidth and differ only in write bandwidth. The traces are replayed against two models, Llama-4-Maverick and Qwen3-235B-A22B. Each is served across eight GPUs, its attention by data parallelism and its experts by expert parallelism. Every configuration is run at two SLOs on the multi-agent trace, both with a 2 s time to first token (TTFT) [42, 50]. They differ in the 99th-percentile time per output token (TPOT), a tight 60 ms and a loose 100 ms [6, 42]. The single-agent traces are run at the tight SLO alone.

Workloads. Three replay traces are built from the sessions of §3.2 (Table 2). The temporal regime supplies the two single-agent traces, τ^3 -bench and BFCLv4, and the structural regime’s five benchmarks are mixed into one multi-agent trace. The multi-agent trace is collected twice, once as the harness records it and once with the reviewer disabled, and every result below is read on the first unless it names the second (§5.4). The evaluation uses the latency model of §3.2, and both of its terms are taken from prior serving work. No benchmark records when a user replies, so user delay is sampled from a log-normal with median 120 s [12, 49, 54], and tool-call latency from a published measurement for each tool class (Appendix B).

Baseline. Lachesis is compared against HBM-first placement, which places a KV block in HBM whenever HBM has a free block and in HBF otherwise. It is the placement an engine performs without Lachesis.

Metric. HBF lifetime is the time HBF’s P/E budget lasts at the mean rate at which KV is written to it, excluding the one write that loads the model weights. Each cell tolerates 100,000 P/E cycles at SLC density [41], over 512 GB of flash per stack [38]. KV blocks are written whole and in order, so garbage collection and wear leveling move few blocks and write amplification is 1.02, the value prior work adopts [16, 23]. Lifetime is measured once the write traffic has settled. A device must outlast its five-year warranty.

5.3 Multi-Agent Results

Overall results. At a tight SLO, HBF outlasts the five-year warranty under Lachesis at every GPU-memory configuration and on both models, and under HBM-first placement only with Qwen3-235B-A22B on H3’s daisy chain (Figure 9, top row). At the four configurations that give HBM one to four of the eight slots, Lachesis reaches 3.3–9.3 device-years against HBM-first’s 1.9–4.8, extending HBF lifetime by 1.50–2.95 $\times$ at the tight SLO and 1.19–1.89 $\times$ at the loose one. Lachesis extends HBF lifetime over HBM-first at every model, SLO and configuration of the multi-agent trace.

What sets the gain. Lachesis’s gain over HBM-first is set by one ratio, HBM’s capacity against the short-lived KV live at one time, and the GPU-memory configuration and the SLO

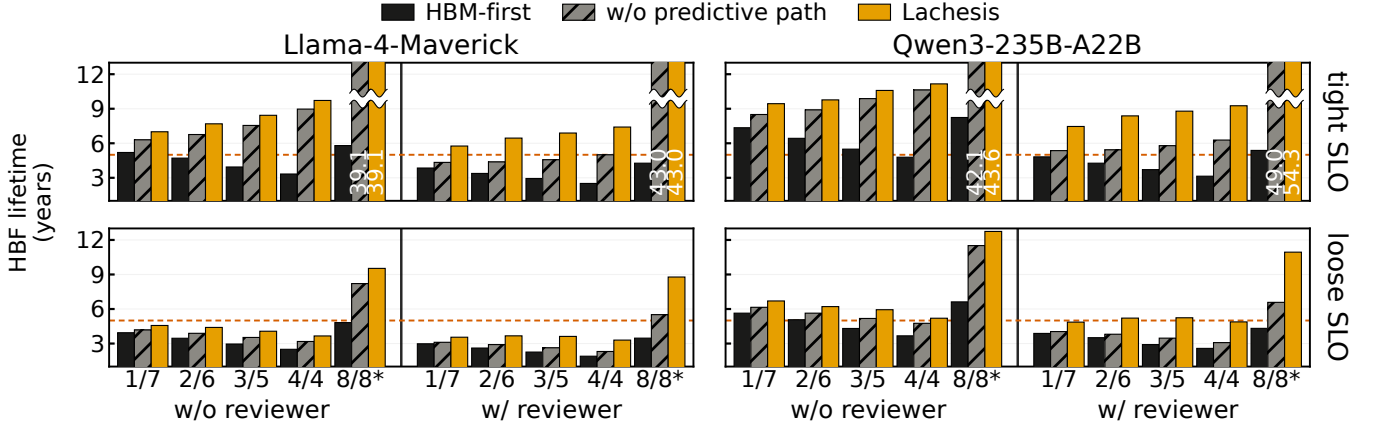


Figure 9. HBF lifetime in device-years on the multi-agent trace. The dashed line is the five-year warranty.

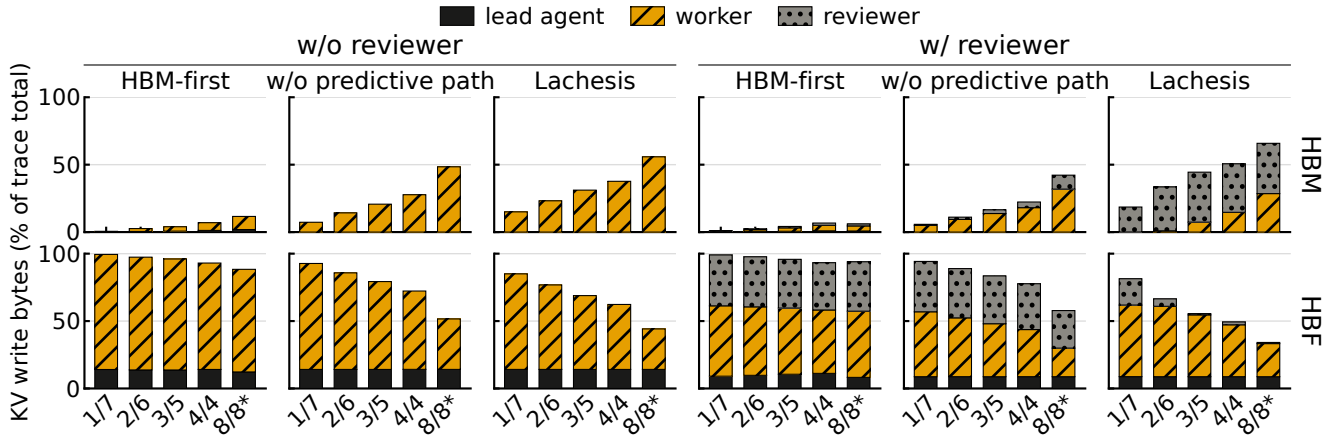


Figure 10. KV write bytes by segment in each tier on the multi-agent trace, with Llama-4-Maverick at the loose SLO.

each move it. Moving from one HBM slot in eight to four quadruples HBM capacity and removes three of the seven HBF stacks, so the aggregate P/E budget falls with them. At the tight SLO, the added HBM absorbs 12% of HBM-first’s flash writes and 54–56% of Lachesis’s. Only the second offsets the smaller P/E budget, so HBF lifetime falls under HBM-first and rises under Lachesis with each slot added (Figure 9, top row). The longest-lived configuration is therefore the smallest HBM that absorbs most of the live short-lived KV, because further slots cost more P/E budget than they save in flash writes. A looser SLO instead admits more concurrent sessions, so that ratio falls and the gain narrows (Figure 9, bottom row). It also moves the longest-lived configuration down to two or three HBM slots in eight.

Write composition. Lachesis writes 8.2–19.5× as much KV to HBM as HBM-first placement, so HBF receives less and spends less of its P/E budget (Figure 10). HBM takes 19–51% of a run’s write bytes under Lachesis against 1–7% under HBM-first, and the rest of that volume reaches HBF. HBM-first frees blocks only under memory pressure, so its

HBM capacity does not turn over. Lachesis places the lead-agent segment in HBF at the write and admits the workers predicted to be short-lived, so HBM’s capacity turns over. The reviewer increases the short-lived KV live at one time, and HBM’s capacity does not grow with it, so more of that KV reaches HBF and HBF lifetime falls under both placement policies. Lachesis places that KV in HBM by its segment class, so the loss is smaller under Lachesis than under HBM-first (§5.4).

H3’s daisy chain. Lachesis extends HBF lifetime by 10.06–10.09× at the tight SLO on H3’s daisy chain, the largest gain at any configuration. H3 does not trade HBM capacity against flash stacks, because its eight slots carry HBM and HBF together instead of dividing them, so HBM grows and no flash stack is lost (Figure 9, 8/8*). The gain falls to 2.53× on the change of SLO alone.

5.4 Ablation Study

Lachesis’s gain over HBM-first is the product of two margins, one from each placement path (§5.3). Lachesis *without the*

predictive path places every segment by the segment class the harness fixes and ranks no worker. It yields $1.11\text{--}2.00\times$ over HBM-first on the multi-agent trace, and Lachesis adds $1.33\text{--}1.54\times$ over it (Figure 9). Both margins hold on both models and at every configuration that splits the eight slots. **What the reviewer changes.** The predictive path ranks only worker segments, and what it earns is set by the HBM capacity that ranking keeps free for the reviewer, $1.33\text{--}1.54\times$ with a reviewer present against $1.05\text{--}1.14\times$ without one (Figure 9). A reviewer’s segment is generally shorter-lived than a worker’s, so Lachesis places it in HBM by its segment class with or without the predictive path (§4). Without the ranking, long-lived worker segments enter HBM and hold its capacity until their workers return. Lachesis instead admits the worker segments predicted to be short-lived, which are also the ones that write less (§3.5), so the same HBM covers more of the short-lived KV live at one time and usually has room for a reviewer. The reviewer’s own write volume reaches flash at 45–95% without the predictive path, and at 2–25% under Lachesis (Figure 10). In the sessions with no reviewer, workers are the only segments HBM holds, so ranking them only changes which of them it holds.

5.5 Single-Agent Results

Overall results. HBF outlasts the five-year warranty under Lachesis at every configuration but one, and under HBM-first placement at some of τ^3 -bench’s and none of BFCLv4’s (Figure 11). At the four configurations that give HBM one to four of the eight slots, Lachesis reaches 4.6–12.2 device-years against HBM-first’s 2.1–6.4, extending HBF lifetime by $1.35\text{--}3.13\times$. These traces are single-agent, so they exercise the temporal axis alone and the deterministic path places every segment (§4).

What a larger HBM admits. Lachesis’s lifetime peaks at one to three HBM slots in eight and falls with every slot added past the peak (Figure 11). A larger HBM admits no more KV once it holds every reasoning-laden context segment the deployment has live at one time, and its share of a run’s write bytes flattens (Figure 12). Past the peak, every added slot costs an HBF stack and saves no flash write, and Lachesis reaches 30–31% fewer device-years on BFCLv4 at four HBM slots in eight than at one. Under Lachesis, H3’s daisy chain loses no flash stack and reaches the longest lifetime at every benchmark and model. The multi-agent trace peaks later at the same SLO, because its worker segments give Lachesis more to admit than HBM has room for (§5.3).

6 Related Work

Lachesis places KV in flash by the lifetime of a context segment, which no HBF design for LLM inference has used to decide what flash holds. Early designs place only read-only data. H3 [14] stores model weights and precomputed KV in HBF and keeps generated KV in HBM, and FLINT [29]

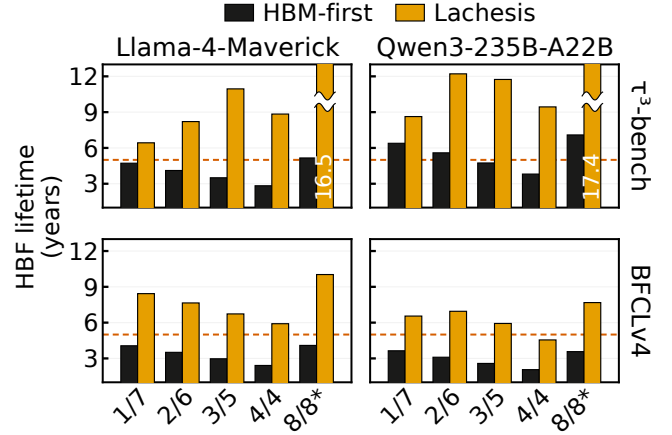


Figure 11. HBF lifetime in device-years on the single-agent traces. The dashed line is the five-year warranty.

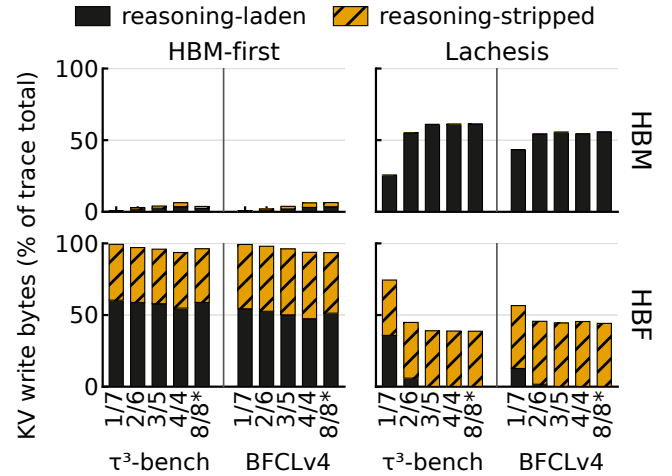


Figure 12. KV write bytes by segment in each tier on the single-agent traces, with Qwen3-235B-A22B at the tight SLO.

replaces the flash translation layer with a read-only one, which locates model weights and supports no writes. Neither design spends flash endurance on generated KV, because neither writes it. Later designs do write generated KV, and each answers the write cost with a mechanism. FlashAccel [47] places weights and KV together in flash, Park et al. [33] offload a buffered write-back into a custom base die, and DASH [20] holds decode KV in HBM until it can be written to flash in whole pages. These designs lower what each write costs, and DASH also decides where data goes, from its size, mutability and reuse. None of the three decides from how long the data stays readable. Son et al. [41] measure what writing every generated KV block to flash costs. When HBF serves as the main GPU memory, each flash block accumulates more P/E cycles than SLC NAND tolerates. Kyung et al. [23] show that one property can decide which KV is worth that cost, a request’s output length (§2.3). Lachesis instead

decides from a lifetime the agent harness fixes before the segment is written.

7 Conclusion

This paper presents Lachesis, a lifetime-aware KV cache placement layer for agent serving across HBM and HBF. We find that the context an agent session accumulates divides at harness-defined context boundaries into context segments whose lifetimes differ along three axes, temporal, structural and inter-worker, and that each lifetime is knowable before its segment is written. Lachesis places each segment on the tier its lifetime implies, along a deterministic path that reads the segment class the harness fixes and a predictive path that ranks worker segments at spawn. In trace-driven simulation, Lachesis extends HBF lifetime by 1.19–3.13× over HBM-first placement, and HBF outlasts a five-year warranty on the multi-agent trace at a tight SLO even under 24×7 operation.

Acknowledgments

This paper was the result of the research project supported by SK hynix Inc. Jae W. Lee is the corresponding author. This work used Claude Code for code, language editing and text refinement. All technical content, experiments, and conclusions were verified by the authors.

References

- [1] 2025. LLMsSimulator. Source repository. <https://github.com/scale-snu/LLMsSimulator>
- [2] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiyang Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. <https://proceedings.mlr.press/v235/abhyankar24a.html>
- [3] Hyowon An, Gyeongjae Lee, Zhixian Jin, Joungho Kim, and John Kim. 2026. Memory Fabric: HBM Die-to-Die Interconnect for GPU Bandwidth Expansion. *IEEE Computer Architecture Letters* 25, 2 (2026), 327–330. doi:10.1109/LCA.2026.3721758
- [4] Anthropic. 2026. How we built our multi-agent research system. Anthropic Engineering. Accessed 22 August 2026. <https://www.anthropic.com/engineering/multi-agent-research-system>
- [5] Anthropic. 2026. Subagents. Claude Code documentation. Accessed 22 August 2026. <https://docs.claude.com/en/docs/claude-code/sub-agents>
- [6] Yukang Chen, Weihao Cui, Han Zhao, Ziyi Xu, Xiaozhe Fan, Xusheng Chen, Yangjie Zhou, Shixuan Sun, Bingsheng He, and Quan Chen. 2026. Towards High-Goodput LLM Serving with Prefill-decode Multiplexing. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. doi:10.1145/3779212.3790236
- [7] Rongxin Cheng, Yuxin Lai, Xingda Wei, Rong Chen, and Haibo Chen. 2026. KunServe: Parameter-centric Memory Management for Efficient Memory Overloading Handling in LLM Serving. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys)*. doi:10.1145/3767295.3769348
- [8] Eunyeong Cho, Jehyeon Bang, Ranggi Hwang, and Minsoo Rhu. 2026. PASCAL: A Phase-Aware Scheduling Algorithm for Serving Reasoning-based Large Language Models. In *Proceedings of the 32nd IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. <https://arxiv.org/abs/2602.11530>
- [9] DeepSeek. 2026. Thinking Mode. DeepSeek API documentation. Accessed 22 August 2026. https://api-docs.deepseek.com/guides/thinking_mode
- [10] Jingqi Feng, Yukai Huang, Rui Zhang, Sicheng Liang, Ming Yan, and Jie Wu. 2025. WindServe: Efficient Phase-Disaggregated LLM Serving with Stream-based Dynamic Scheduling. In *52nd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA '25)*. 1283–1295. doi:10.1145/3695053.3730999
- [11] Yicheng Feng, Xin Tan, Yangtao Deng, Yimin Jiang, Yibo Zhu, and Hong Xu. 2026. Frontier: Towards Comprehensive and Accurate LLM Inference Simulation. arXiv:2605.21312 <https://arxiv.org/abs/2605.21312>
- [12] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast State Restoration in LLM Serving with HCache. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 128–143. doi:10.1145/3689031.3696072
- [13] Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. 2026. DeepSearchQA: Bridging the Comprehensiveness Gap for Deep Research Agents. arXiv:2601.20975 <https://arxiv.org/abs/2601.20975>
- [14] Minh Ha, Euseok Kim, and Hoshik Kim. 2026. H3: Hybrid Architecture Using High Bandwidth Memory and High Bandwidth Flash for Cost-Efficient LLM Inference. *IEEE Computer Architecture Letters* 25, 1 (2026), 49–52. doi:10.1109/LCA.2026.3660969
- [15] Harbor Framework Team. 2026. Harbor: A framework for evaluating and optimizing agents and models in container environments. Version v0.22.0. doi:10.5281/zenodo.20953922
- [16] Inho Jeong, Sunghyeon Woo, Sol Namkung, and Dongsuk Jeon. 2025. HiFC: High-efficiency Flash-based KV Cache Swapping for Scaling LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=onhjdWCxZY>
- [17] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations (ICLR)*. 54107–54157. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html
- [18] Yunho Jin, Chun-Feng Wu, David Brooks, and Gu-Yeon Wei. 2023. S³: Increasing GPU Utilization during Generative Inference for Higher Throughput. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/3a13be0c5dae69e0f08065f113fb10b8-Abstract-Conference.html
- [19] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Petros Vavaroutos, Sotirios Xydis, and Dimitrios Soudris. 2025. throttLL'eM: Predictive GPU Throttling for Energy Efficient LLM Inference Serving. In *Proceedings of the 31st IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. doi:10.1109/HPCA61900.2025.00103
- [20] Seeyeon Kim, Juhyeong Jin, and Joo-Young Kim. 2026. Beyond Capacity: Scalable MoE LLM Inference via High-Bandwidth Flash with Direct GPU and HBM Paths. arXiv:2608.14333 [cs.AR] doi:10.48550/arXiv.2608.14333
- [21] Kimi Team. 2026. Kimi K2.5: Visual Agentic Intelligence. arXiv:2602.02276 <https://arxiv.org/abs/2602.02276>
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. ACM, 611–626. doi:10.1145/3600006.3613165

- [23] Kwanhee Kyung, Yeon Ji Moon, Juhwan Cho, and Jung Ho Ahn. 2026. High-Bandwidth Flash for KV Caches: Endurance and Performance Implications. *IEEE Computer Architecture Letters* 25, 1 (2026), 210–213. doi:10.1109/LCA.2026.3695938
- [24] LangChain. 2026. deepagents. Source repository, commit 23b83ad. Accessed 22 August 2026. <https://github.com/langchain-ai/deepagents/blob/23b83ad50f63d241d0069a3dc426d43b211adf2e/README.md>
- [25] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Rui Zhang, Kuntai Du, et al. 2025. LMCACHE: An efficient KV cache layer for enterprise-scale LLM inference. arXiv:2510.09665 [cs.LG] doi:10.48550/arXiv.2510.09665
- [26] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 32332–32344. <https://proceedings.mlr.press/v235/liu24bz.html>
- [27] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walsh, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. 2026. Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. arXiv:2601.11868 [cs.SE] <https://arxiv.org/abs/2601.11868>
- [28] Piotr Nawrot, Adrian Łańcucki, Marcin Chochowski, David Tarjan, and Edoardo Ponti. 2024. Dynamic Memory Compression: Retrofitting LLMs for Accelerated Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 37396–37412. <https://proceedings.mlr.press/v235/nawrot24a.html>
- [29] Geraldo F. Oliveira, Xiangyu Zhu, Ahmet Caner Yüzügüler, Vamanan Arulchelvan, Lukas Cavigelli, Luo Longfei, Jia Xinglei, Onur Mutlu, Zhou Ke, Shai Bergman, Arash Tavakkol, and Ji Zhang. 2026. FLINT: Efficiently Exploiting High-Bandwidth Flash for Capacity-Scalable LLM Inference Acceleration. *IEEE Computer Architecture Letters* (2026). doi:10.1109/LCA.2026.3729746
- [30] OpenAI. 2026. Agent orchestration. OpenAI Agents SDK documentation. Accessed 22 August 2026. https://openai.github.io/openai-agents-python/multi_agent/
- [31] OpenAI. 2026. Better performance from reasoning models using the Responses API. OpenAI Cookbook. Accessed 22 August 2026. https://developers.openai.com/cookbook/examples/responses_api/reasoning_items
- [32] OpenAI. 2026. Reasoning models. OpenAI API documentation. Accessed 22 August 2026. <https://developers.openai.com/api/docs/guides/reasoning>
- [33] Junho Park, Hyowon An, Haeseok Suh, Youngsu Yoon, Hyuni Lee, and Joungho Kim. 2026. HBM-HBF-Centric Memory Pooling Architecture With Custom Base Die for Terabyte-Scale LLM Inference. *IEEE Computer Architecture Letters* 25, 2 (2026), 259–262. doi:10.1109/LCA.2026.3703982
- [34] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *51st ACM/IEEE Annual International Symposium on Computer Architecture (ISCA '24)*. 118–132. doi:10.1109/ISCA59077.2024.00019
- [35] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 267)*. PMLR. <https://proceedings.mlr.press/v267/patil25a.html>
- [36] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2026. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Transactions on Storage* 22, 4 (2026), 1–38. doi:10.1145/3773772
- [37] Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. 2026. AOrchestra: Automating Sub-Agent Creation for Agentic Orchestration. arXiv:2602.03786 <https://arxiv.org/abs/2602.03786>
- [38] Sandisk Corporation. 2025. Sandisk Unveils the Future of Memory Architecture for AI. Introducing: High Bandwidth Flash. Tech brief, HBF Fact Sheet. https://documents.sandisk.com/content/dam/asset-library/en_us/assets/public/sandisk/collateral/company/Sandisk-HBF-Fact-Sheet.pdf
- [39] Sierra Research. 2026. τ^3 -bench, release v1.0.1. <https://github.com/sierra-research/tau2-bench/releases/tag/v1.0.1>
- [40] SK hynix Inc. 2026. SK hynix Unveils First HBF Standard Specifications with Sandisk, Presenting AI Memory Solutions at FMS 2026. SK hynix Newsroom. <https://news.skhyunx.com/en/hbf-at-fms-2026/>
- [41] Downon Son, Yonggon Park, Hyunuk Cho, Hyungkyu Ham, Onur Mutlu, Sungjin Lee, Gwangsun Kim, and Jisung Park. 2026. Exploring High-Bandwidth Flash for Modern LLM Inference: Opportunities and Challenges. *IEEE Computer Architecture Letters* 25, 2 (2026), 251–254. doi:10.1109/LCA.2026.3705817
- [42] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *Proceedings of the 31st IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. doi:10.1109/HPCA61900.2025.00102
- [43] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. QUEST: Query-Aware Sparsity for Efficient Long-Context LLM Inference. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 47901–47911. <https://proceedings.mlr.press/v235/tang24l.html>
- [44] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024*. Association for Computational Linguistics, 16022–16076. doi:10.18653/v1/2024.acl-long.850
- [45] vLLM Project. 2026. vLLM v1 KV-cache block management (vllm/v1/core/kv_cache_utils.py). Source repository, release v0.27.1, commit 6e448d0. https://github.com/vllm-project/vllm/blob/v0.27.1/vllm/v1/core/kv_cache_utils.py
- [46] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen,

- Robert Brennan, and Graham Neubig. 2025. The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. arXiv:2511.03690 [cs.SE] <https://arxiv.org/abs/2511.03690>
- [47] Xinyu Wang, Yalong Xue, Xiaotian Sun, Xiaoyu Zhang, Xinjiang Zhang, Chunmeng Dou, Xueqi Li, and Xiaoming Chen. 2026. FlashAccel: Leveraging High-Bandwidth Flash (HBF) for High-Throughput LLM Inference. arXiv:2607.10186 [cs.AR] doi:10.48550/arXiv.2607.10186
- [48] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient Streaming Language Models with Attention Sinks. In *The Twelfth International Conference on Learning Representations (ICLR)*. 21875–21895. https://proceedings.iclr.cc/paper_files/paper/2024/hash/5e5fd18f863cbe6d8ae392a93fd271c9-Abstract-Conference.html
- [49] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. 2026. Strata: Hierarchical Context Caching for Long Context Language Model Serving. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*. USENIX Association, Seattle, WA, 1–16. <https://www.usenix.org/conference/osdi26/presentation/xie-zhiqiang>
- [50] Chuhao Xu, Zijun Li, Quan Chen, Han Zhao, Xueyan Tang, and Minyi Guo. 2026. Towards Resource-Efficient Serverless LLM Inference with SLInfer. In *Proceedings of the 32nd IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. <https://arxiv.org/abs/2507.00507>
- [51] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv:2406.12045 <https://arxiv.org/abs/2406.12045>
- [52] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=WE_vluYUL-X
- [53] Naimeng Ye, Arnab Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. 2026. Speculative Actions: A Lossless Framework for Faster AI Agents. In *Proceedings of the Fourteenth International Conference on Learning Representations (ICLR)*. <https://iclr.cc/virtual/2026/poster/10009726>
- [54] Lingfan Yu, Jinkun Lin, and Jinyang Li. 2025. Stateful Large Language Model Serving with Pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. ACM, 144–158. doi:10.1145/3689031.3696086
- [55] Yichao Yuan, Ankita Nayak, Souvik Kundu, and Nishil Talati. 2026. Agentic AI Workload Characteristics. arXiv:2605.26297 [cs.DC] doi:10.48550/arXiv.2605.26297
- [56] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. BlitzScale: Fast and Live Large Model Autoscaling with O(1) Host Caching. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. <https://www.usenix.org/conference/osdi25/presentation/zhang-dingyan>
- [57] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. Curran Associates, Inc., 34661–34710. doi:10.52202/075280-1506
- [58] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. doi:10.52202/079017-2000
- [59] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 193–210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>
- [60] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *The Twelfth International Conference on Learning Representations (ICLR)*. 15585–15606. https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html

A Workload Corpus Statistics

Harness configuration. The temporal regime runs each benchmark under Harbor’s own official prompts. Each structural-regime benchmark’s official prompts and reference agent implementation are ported to AOrchestra.

Token columns. Tokens processed counts every request’s input and output, so the context a round reuses is counted again at each round. Tokens written counts only what each request newly materializes, which is the quantity §3.3 reports write volume over. The structural traces record per-segment sizes rather than per-request lengths, so the table reports only the second column for them.

No session is excluded from either corpus, including those that failed or timed out.

B Trace Fields, Clock Charges, and KV Accounting

This appendix gives the trace fields, the clock charges and the exact $\rho_{R/W}$ accounting that §3.2 states in one sentence each, for reproducibility [23].

The trace records, for each request, its session, turn, and sub-agent lineage, its tokenized input and output, and its tool-call and observation boundaries, alongside the two cache fields §3.2 names. With the trace aligned in session order, the matched prefix gives the KV reused from earlier requests, and the unmatched suffix plus the generated tokens gives the KV newly materialized. A segment built over several requests becomes live at its first write.

Appending a tool result forces a partial prefill before the next round can decode. The clock of §3.2 charges TTFT_{ref} once per observation, whatever that observation’s length. A prefill deployment provisioned to the reference point completes the prefill for any single observation within that time. $\Lambda_{\text{tool}}(s)$ is drawn from an exponential distribution [53] whose mean is the published measurement for the calling tool’s class [2], at the granularity each regime’s trace records tool identity.

Let $V_{\text{KV}}^{\text{token}}$ be the per-token KV footprint of the evaluated attention architecture, and let $N_W(s)$ be the number of tokens whose KV state is materialized for segment s at the

Table 3. Corpus scale per workload.

Regime	Workload	Sessions	Requests	Tokens processed	Tokens written
Temporal	τ^3 -bench airline	58	709	4,735,468	591,248
	τ^3 -bench retail	114	1,387	9,593,474	1,156,112
	τ^3 -bench telecom	114	2,208	20,687,494	1,407,427
	τ^3 -bench banking	97	2,221	63,759,218	7,172,200
	BFCLv4 base	200	2,063	8,865,141	800,193
	BFCLv4 long-context	200	2,066	23,312,059	5,279,265
Structural	AppWorld	168	6,248	—	9,738,267
	DeepSearchQA	125	12,241	—	37,196,143
	SWE-bench Verified	149	8,745	—	11,677,326
	TerminalBench	75	3,675	—	7,289,834
	WebArena	106	2,190	—	6,707,151

decode node. Then

$$V_W(s) = N_W(s) V_{KV}^{\text{token}}. \quad (2)$$

For request q with output length $L_{\text{out}}(q)$, let $n_s(q, i)$ be the number of KV tokens from s present before its i -th decode step. Under full-context attention,

$$V_R(s) = V_{KV}^{\text{token}} \sum_{q \in Q(s)} \sum_{i=1}^{L_{\text{out}}(q)} n_s(q, i), \quad \rho_{R/W}(s) = \frac{V_R(s)}{V_W(s)}, \quad (3)$$

where $Q(s)$ is the set of requests whose decode stage reads s .

For a stateless request this reduces to Kyung et al.’s request-level expression, under their decode-stage accounting but computed per segment. Only decode-stage reads count: observation-induced partial prefill is counted in T_{norm} through the TFT_{ref} term, not $V_R(s)$. Initial prefill tokens are counted in $V_W(s)$, while initial prefill time is not counted in T_{norm} .